# CSE 3320: Operating Systems

# Topic 4: File System Management

Jiayi Meng

http://ranger.uta.edu/~jmeng/

# Outline

## Lecture 1 — How a request travels, and what it costs

- I/O subsystem: buses, drivers, device registers, polling vs. interrupts, PIO vs. DMA
- File descriptors and the open file table: what your program actually holds
- Disks: geometry, the timing model, scheduling — and why flash breaks all of it

## Lecture 2 — Naming and finding data

- Files, directories, and on-disk structures: superblock, inode, links
- Path lookup, and disk allocation: contiguous, linked, indexed, multi-level, extents

## Lecture 3 — Making it fast, and making it survive a crash

- Caching and the Fast File System
- Crash consistency, journaling, RAID, and checksums

# I/O Devices: What the OS Has to Manage

- I/O devices of many flavors
  - Storage devices: Disk
  - Human-interface devices: Mouse, keyboard
  - Transmission devices: NIC (Network Interface Card)
  - Clocks and timers

- They differ in **speed** (nanoseconds to milliseconds), **unit of transfer** (a byte, a block, a packet), **direction**, and **who starts the conversation**
- Yet your program writes the same `read()` and `write()` for all of them

- Three questions drive this lecture:
  1. How does one OS work with devices it has never seen? *(Part 1)*
  2. How does your program reach that machinery, and what happens while it waits? *(Parts 2–3)*

# The Crux of the Problem: How to Build a Device-Neutral OS?

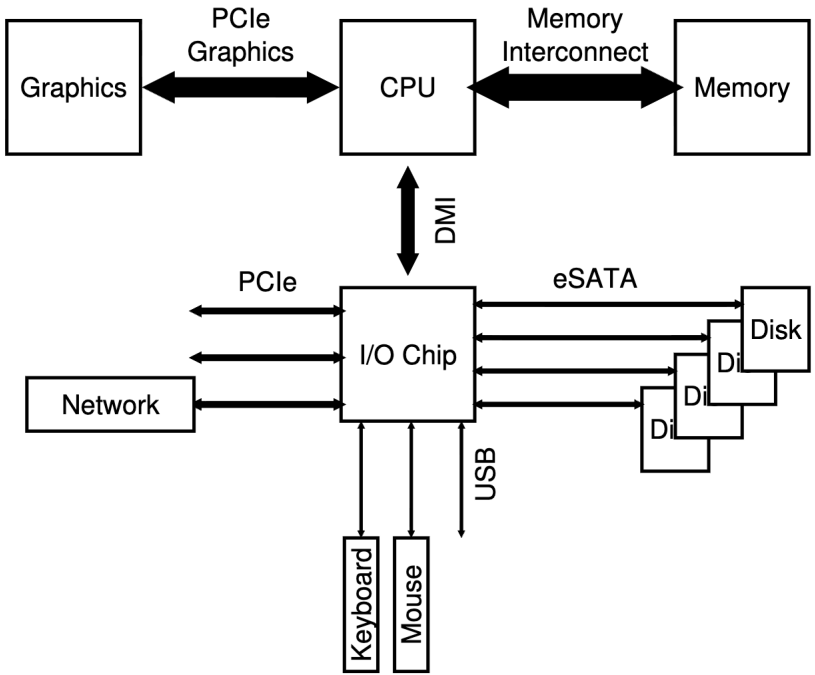
- How can we keep most of the OS device-neutral, thus hiding the details of device interactions from major OS subsystems?
- Key idea: Device driver + I/O subsystem management
  - A software in the OS that must know in detail how a device works
  - Any specifics of device interaction are encapsulated within it
    - It is needed for every device that is plugged into a system

- Studies of the Linux kernel find that **over 70% of OS code is device drivers**
  - The device-neutral core is the small part. The translation layer is the bulk.

# Part 1 · The abstraction

How does one OS work with devices it has never seen?

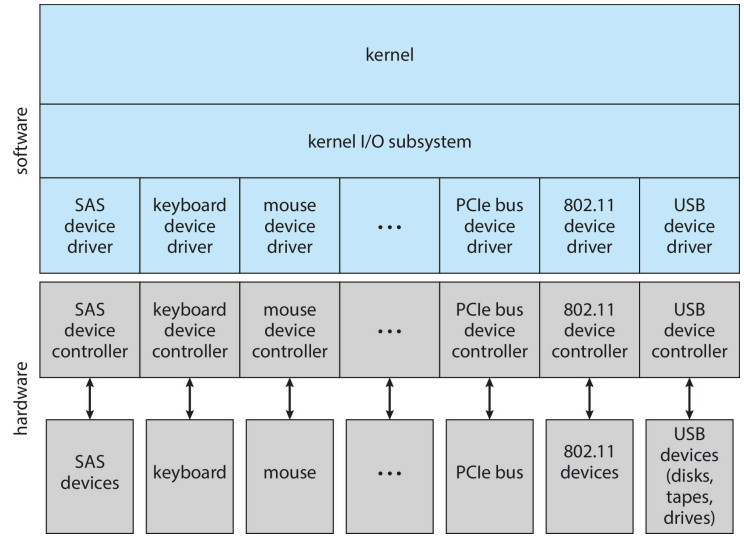
# Modern System Architecture




- Devices are not all attached the same way — they sit on a **hierarchy of buses**
  - **CPU–memory bus**: fastest; CPU to RAM, directly
  - **PCIe / general I/O bus**: fast devices — GPU, NVMe SSD, NIC
  - **Peripheral bus** (SATA, USB): slowest — disks, keyboards, mice
- Faster buses sit **physically closer** to the CPU, and cost more per pin
- A device's speed tier and its bus tier match
  - NVMe SSD → PCIe · hard disk → SATA · mouse → USB
- A hard disk is therefore slow **twice over**: slowest bus tier, *and* mechanical delay on top

# I/O Hardware: Which Device Sits Where

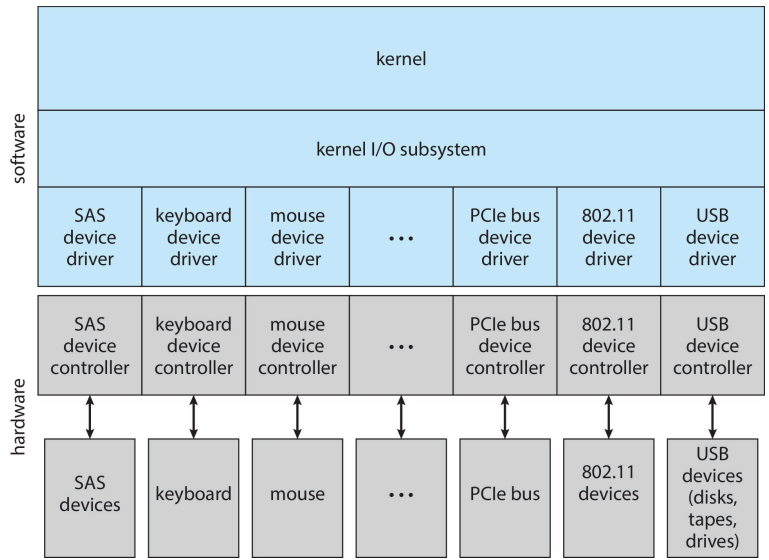
- A real machine's I/O plumbing: PCIe for bandwidth, SAS/SATA for storage, USB for peripherals




- Every one of these speaks a **different command language**, and new ones ship every year
- You cannot rewrite the OS once per device — that is what the next slide fixes

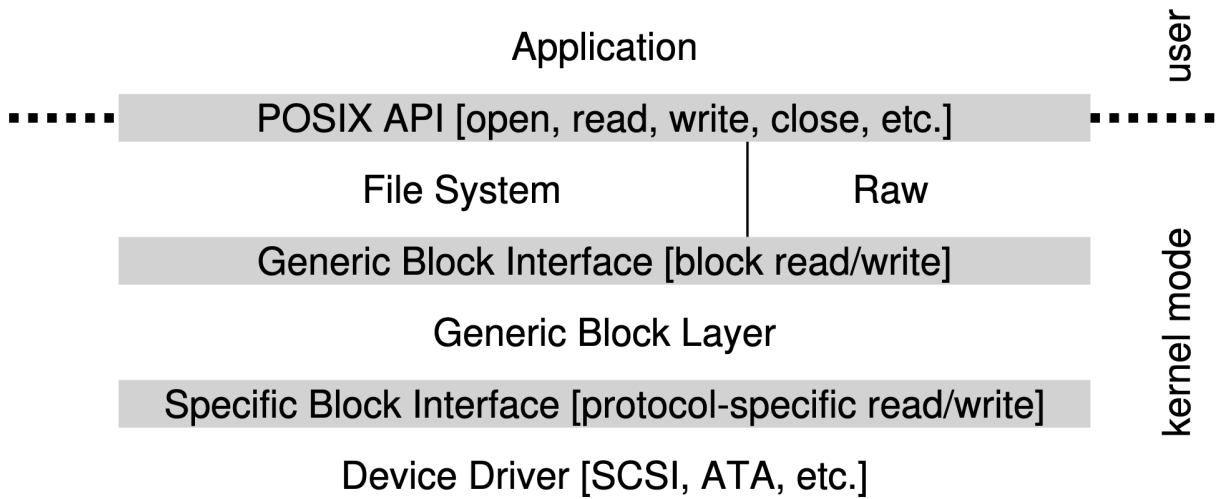
# Device Driver + I/O Subsystem Mgmt.

- The driver presents a **standard interface upward** and speaks the **device's own language downward**
- One driver per device type; the kernel above sees only the standard interface
- Over 70% of Linux kernel source is device drivers — that is the price of this abstraction



# System Stack

- Application → POSIX API → file system → **generic block layer** → specific driver → device
- Everything **above** the generic block layer is device-neutral; everything below is device-specific
- A file system issues "read block N" and never learns whether N lives on SATA, SAS, or NVMe



# Key Generic I/O Interfaces

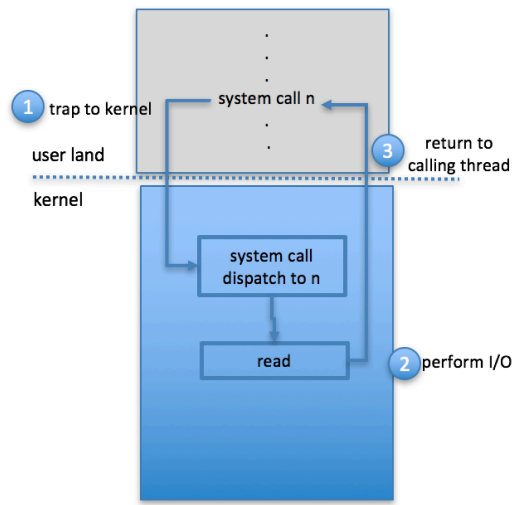
- Storage devices: read(), write()
- Human-interface devices (mouse, keyboard) Character devices: get(), put()
  - Libraries layered on top allow line editing
- Transmission devices: read(), write()
- Clocks and timers: ioctl()
- Notice how few there are. The OS deliberately offers a **narrow** interface
  - A narrow interface is the cost of device-neutrality: special capabilities go unused
  - `ioctl()` is the escape hatch for everything that does not fit

# Part 2 · Going down

How does your program reach that machinery, and what does it hold onto?

# System Call to Perform I/O

- Your program calls `read(fd, buf, n)` → **trap** into the kernel
- Kernel looks up what `fd` refers to, hands a request to the driver, and **blocks** the process
- Driver programs the device registers; the scheduler runs somebody else
- Device finishes → **interrupt** → handler copies data and wakes the process



# File Descriptor

- File descriptor (fd): An integer used to represent a file in a process
  - Easier than using names
- Each process contains a *file descriptor table* for all of its opened file descriptors
- File Descriptor Table

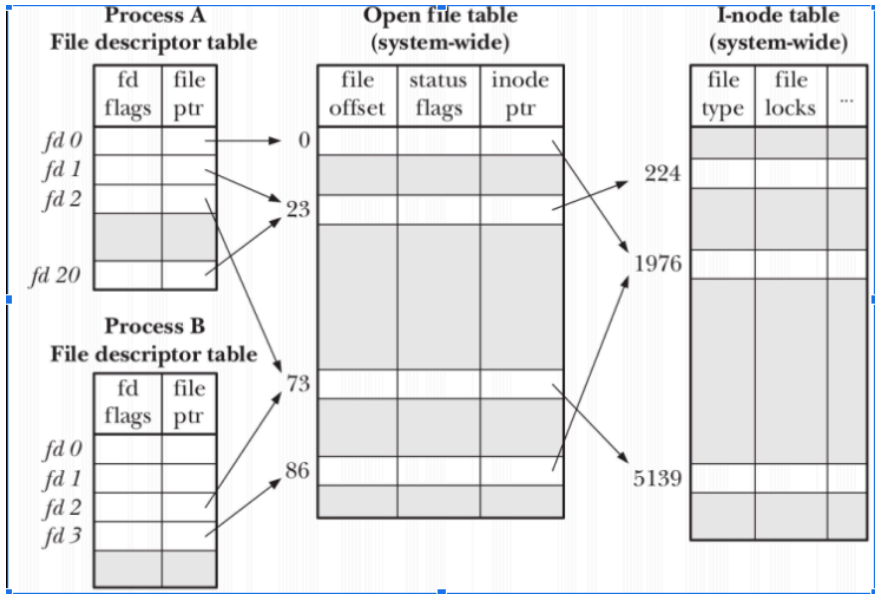
```
// the information xv6 tracks about each process
// including its register context and state
struct proc {
  char *mem;               // Start of process memory
  uint sz;                 // Size of process memory
  char *kstack;            // Bottom of kernel stack
                           // for this process
  enum proc_state state;   // Process state
  int pid;                 // Process ID
  struct proc *parent;     // Parent process
  void *chan;              // If !zero, sleeping on chan
  int killed;              // If !zero, has been killed
  struct file *ofile[NOFILE]; // Open files
  struct inode *cwd;       // Current directory
  struct context context; // Switch here to run process
  struct trapframe *tf;    // Trap frame for the
                           // current interrupt
};
```

# Open File Table

- In-memory, system-wide list of file metadata *in use*
- Each file table entry contains
  - Current file offset
    - Which part of the file the next read or write will access
  - Status flags
  - Inode pointer (ptr)
  - Reference count
    - When a file table entry is shared by another process, increment its reference count
    - Only when both processes close the file (or exit) will the entry be removed

# File Descriptor & Open File Table

- File Descriptor Table
  - in PCB — per process
- Open File Table
  - system-wide; holds the **offset**
- Inode Table
  - the file's on-disk record
  - treat it as a black box today — we open it next lecture




Adapted from https://man7.org/training/download/lusp_fileio_slides.pdf

# fork() and File Descriptor Inheritance

- When a process calls `fork()`, the child **inherits** all of the parent's open file descriptors
  - The child's fd table is a copy: same fd integers, same open file table entries
  - The **reference count** in each shared open file table entry is incremented
- Parent and child share the **same file offset** for inherited fds
  - A write by the child advances the offset seen by the parent, and vice versa
- `close()` in the child only decrements the reference count — it does not close the file for the parent
  - The open file table entry is removed only when reference count reaches 0

# Blocking and Nonblocking I/O

- **Blocking I/O**: the calling process sleeps until the I/O operation completes
  - Simple to program; process makes no progress while waiting
  - OS moves the process to the **Blocked** state; another process runs in the meantime

- **Non-blocking I/O**: the system call returns immediately, even if I/O is not done
  - Returns partial data or an error code (e.g., `EAGAIN`) if not yet ready
  - Process must check back later (polling loop) or be notified via a callback

- **Asynchronous I/O**: process is notified (signal or callback) when operation finishes
  - Allows overlap of computation and I/O without manually re-checking

- "The process blocks" is not a new mechanism — it is the scheduler from Topics 1 and 3
  - The OS moves the process **Running → Blocked** (Topic 1's process states), and context-switches away
  - The completion interrupt moves it **Blocked → Ready**, so the scheduler can pick it up again

# Part 3 · Coming back

How does the driver talk to the device — and learn that it is done?

# Canonical Device Registers

- Every I/O device exposes a small set of hardware registers to the OS
  - **Status register**: is the device busy, ready, or in error?
  - **Command register**: tells the device what operation to perform (read, write, reset)
  - **Data register**: transfers data between the OS and the device

- The OS talks to a device by reading and writing these registers
  - Write data to the data register, write command to the command register, then poll or wait for an interrupt on the status register

- **Memory-mapped I/O (MMIO)**: device registers appear as memory addresses
  - CPU uses ordinary load/store instructions to read and write device registers
  - No special I/O instructions needed; hardware routes the access to the device

# Polling vs. Interrupts

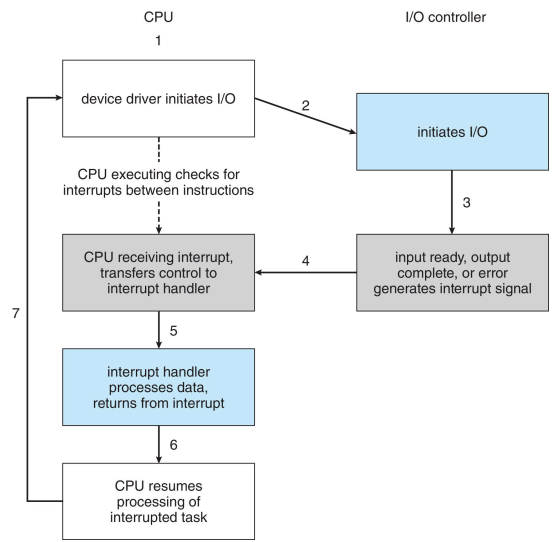
- **Polling**: CPU repeatedly reads the status register until the device is ready
  - Simple but wastes CPU cycles spinning while the device works
  - Good for **very fast devices** (e.g., NVMe SSDs) where the wait is shorter than interrupt overhead

- **Interrupt-driven I/O**: CPU issues request, puts process to sleep, switches to another process
  - Device raises an interrupt when done; OS wakes the sleeping process
  - Eliminates wasted CPU time — good for **slow devices** (spinning disks, USB)

- The right choice depends on device speed relative to interrupt overhead
  - Hybrid: spin briefly first, then fall back to interrupts if not done (**two-phase approach**)

# Interrupt-Driven I/O Cycle

- CPU issues the I/O command, then goes to run another process — **no spinning**
- Device works independently; when finished it raises an **interrupt** on its IRQ line
- CPU stops, saves state, runs the **interrupt handler** (ISR), then resumes what it was doing
- Same mechanism as the timer interrupt from Topic 1 — only the source differs



# Programmed I/O (PIO) vs. DMA

- **Programmed I/O (PIO)**: CPU moves every byte of data between memory and the device
  - CPU writes to the data register one word at a time until the transfer is done
  - Simple, but wastes CPU cycles for large transfers (e.g., writing a 1 MB file)

- **Direct Memory Access (DMA)**: a dedicated DMA controller handles the data transfer
  - CPU tells the DMA controller: source address, destination address, transfer size
  - DMA controller moves data directly between device and memory — CPU is free to do other work
  - Device (or DMA controller) raises an interrupt when the transfer is complete

- DMA is essential for high-throughput I/O
  - Without DMA, moving a large file would monopolize the CPU

# Interrupt Coalescing and Livelock

- **Interrupt coalescing**: device waits briefly and batches multiple completions into one interrupt
  - Reduces total number of interrupts at the cost of a small latency increase
  - Common in high-throughput network cards (NICs) and SSDs

- **Livelock**: too many interrupts overwhelm the OS; user processes never get to run
  - Example: a flooded NIC raises so many interrupts that the OS spends all time in ISRs
  - Symptom: system appears alive (ISRs run) but no useful work is done

- Solution to livelock: **temporarily switch to polling**
  - OS stops accepting interrupts and polls the device directly
  - After the flood subsides, switches back to interrupt-driven mode

# Part 4 · The disk

Once the request arrives, where does the time actually go?

# Hard Disk Drive: Physical Structure

- A hard disk drive (HDD) is a mechanical storage device built from rotating magnetic platters
  - **Platter**: circular disk coated with magnetic material; stores data as magnetic bit patterns
  - **Track**: concentric circle of data on a platter surface; thousands of tracks per surface
  - **Sector**: smallest addressable unit on a track, typically 512 bytes; a disk is an array of sectors numbered 0 to N−1
  - **Disk head**: read/write element on the disk arm; one head per platter surface

- The disk arm moves the head to the correct track, then the platter rotates to bring the target sector under the head
- **Track buffer (cache)**: small on-drive memory (~8–16 MB) that caches recently read tracks
  - The drive reads an entire track at once; subsequent accesses to the same track are fast

- Two of those three things are physical motion. That is the whole story of disk performance.

# Disk I/O Time Model

- Disk I/O time has three components: **T_I/O = T_seek + T_rotation + T_transfer**
  - **Seek time** (T_seek): moving the arm to the correct track; 1–20 ms; most expensive component
    - Phases: accelerate, coast, decelerate, settle
  - **Rotational delay** (T_rotation): waiting for the target sector to rotate under the head
    - Average = half a rotation; ~4 ms at 7200 RPM
  - **Transfer time** (T_transfer): reading or writing the actual sectors; ~0.1 ms for a 4 KB block
- Example for a typical 7200 RPM drive (random 4 KB read):
  - Seek ~5 ms + rotation ~4 ms + transfer ~0.1 ms ≈ **9 ms total**
- Implication: seek and rotation dominate; file systems must minimize seeks

# Disk I/O Time: A Worked Example

- 7200 RPM drive · average seek **5 ms** · transfer rate **100 MB/s** · block size **4 KB**

- **Rotational delay**: one revolution = 60 s / 7200 = **8.33 ms** → on average you wait **half** a revolution = **4.17 ms**
- **Transfer**: 4 KB ÷ 100 MB/s = **0.04 ms** — a rounding error

| Workload | Time | Effective throughput |
|---|---|---|
| One **random** 4 KB read | 5 + 4.17 + 0.04 = **9.21 ms** | ≈ **109 IOPS** → 0.44 MB/s |
| 100 **random** 4 KB reads | 100 × 9.21 = **921 ms** | **0.44 MB/s** |
| 100 **adjacent** 4 KB blocks | 5 + 4.17 + 4.10 = **13.3 ms** | **30.9 MB/s** |

- Same drive, same 400 KB. The only thing that changed is **where the blocks sat**.
- **70× difference.** Every file system design decision in Topic 4 descends from this number.

# Sequential vs. Random I/O: The Performance Gap

- **Sequential I/O**: read blocks in order, one track after another
  - No seek between consecutive blocks; no rotational wait
  - Throughput: ~200 MB/s on a typical 7200 RPM HDD

- **Random I/O**: read blocks at arbitrary locations across the disk
  - Each access incurs full seek + rotation delay (~8–16 ms)
  - Throughput: ~1 MB/s for 4 KB random reads (~100 IOPS)

- The gap: **sequential is ~100–200× faster than random** for HDDs
- This gap is the fundamental constraint on file system design
  - File systems try to store related data sequentially (same track / nearby tracks)
  - Fragmentation (data scattered across disk) turns sequential access into random access

# Disk Scheduling

- The OS can **reorder** pending I/O requests to reduce total seek time
  - Same idea as CPU scheduling in Topic 3 — but the cost being minimized is **arm travel**, not wait time

- Common scheduling policies:
  - **FCFS** (First-Come-First-Served): Service requests in arrival order
    - Fair, but no seek optimization

  - **SSTF** (Shortest Seek Time First): Always service the nearest track next
    - Minimizes seek distance; can **starve** requests for far tracks

  - **SCAN** (elevator algorithm): Sweep head in one direction, service all requests; reverse at end
    - Fair and efficient; avoids starvation

  - **C-SCAN** (Circular SCAN): Sweep in one direction only; return to start without servicing
    - More uniform wait times than SCAN

# Disk Scheduling: A Worked Example

Arm at cylinder **50**. Pending requests: **82, 170, 43, 140, 24, 16, 190**. Cylinders 0–199.

- **FCFS** — serve in arrival order
  - 50 → 82 → 170 → 43 → 140 → 24 → 16 → 190
  - 32 + 88 + 127 + 97 + 116 + 8 + 174 = 642 cylinders
- **SSTF** — always go to the nearest pending request
  - 50 → 43 → 24 → 16 → 82 → 140 → 170 → 190
  - 7 + 19 + 8 + 66 + 58 + 30 + 20 = 208 cylinders
- For comparison: **SCAN** (sweep up to 199, then back) = **332** · **C-SCAN** = **391**
- SSTF travels least — and is the only one of the four that can **starve** a request

# Disk Scheduling: Policy Comparison

| Policy | Strategy | Pros | Cons |
| --- | --- | --- | --- |
| FCFS | Arrival order | Fair | No optimization |
| SSTF | Nearest track | Low seek | Starvation |
| SCAN | Sweep both ways | Fair + efficient | Delay at extremes |
| C-SCAN | Sweep one way | Uniform wait | Return sweep wasted |

# Write-back Caching and Reliability

- Modern drives have an on-drive **write-back cache**: drive reports "write complete" before data reaches the platter
  - Data is in drive memory (volatile) but not yet on magnetic surface
  - A power failure at this point **silently loses data** even though the OS was told it succeeded
- **Write-through caching**: drive only acknowledges write after data is physically on the platter
  - Slower apparent write speed; safe for crash recovery
- File systems that depend on write ordering (e.g., journaling) can be corrupted by write-back caching
  - If the drive reorders writes internally, the file system's carefully ordered journal entries may arrive out of order
- Solution: use `fsync()` to force drive to flush its cache to the platter before continuing
- Remember this slide — it comes back to threaten journaling in Lecture 3

# Part 5 · When the model breaks

What if the device has no moving parts?

# When the Model Breaks: Solid-State Storage

- Everything so far assumed a **moving arm** and a **spinning platter**. Your laptop has neither.
- Flash stores bits in cells with no moving parts — and three operations with wildly different costs:

| Operation | Unit | Typical time |
|---|---|---|
| **Read** a page | 4–16 KB | 10–100 µs |
| **Program** (write) a page | 4–16 KB | 100–500 µs |
| **Erase** a block | 256 KB – 4 MB | 1–5 ms |

- No seek. No rotation. `T_seek + T_rotation + T_transfer` simply does not apply.
- Random reads and sequential reads cost **nearly the same**

# The Flash Constraint: Erase Before Write

- A flash page can be **programmed only if it has been erased first**
- But erase works on an entire **block** — hundreds of pages at once, never a single page

- So overwriting 4 KB in place would mean: read the whole block, erase it, rewrite everything
  - Catastrophically slow, and it burns an erase cycle on every small write

- Flash's answer: never overwrite in place
  - Write the new version to a **fresh, already-erased page**
  - Mark the old page **dead** and move on

- Writes become **append-only**. Internally, an SSD is a **log**.

# The FTL: Making Flash Look Like a Disk

- The OS still wants to say *"write logical block 4096"* — and expects it to land in the same place every time
- The **Flash Translation Layer** (FTL), firmware on the drive, maintains the illusion

- Keeps a map: **logical block number → current physical page**
- Every write goes to a new page; the map entry is updated. The old page becomes **garbage**.
- Dead pages accumulate → **garbage collection**: copy the still-live pages out of a block, then erase it

- This is the same move as the device driver at the start of this lecture
  - Hide the device's real behaviour behind a familiar interface
  - "All computer problems can be solved by another layer of indirection"

# Wear Leveling and Write Amplification

- Each flash block survives a **limited number of erase cycles** — roughly 1,000 to 100,000 depending on cell type
- **Wear leveling**: the FTL deliberately spreads writes across all blocks
  - Without it, a frequently-rewritten file would destroy a few blocks while the rest of the drive stayed pristine
- **Write amplification**: one 4 KB write from the host can cause **far more** than 4 KB of flash writes
  - Garbage collection copies live pages around; those copies are writes too
  - Amplification of 2–10× is routine — it costs both **performance** and **lifetime**
- **TRIM**: the file system tells the drive *"these blocks are deleted, do not preserve them"*
  - Without TRIM the drive faithfully copies data that no file references any more
  - A rare case where the file system must tell the hardware about a **deletion**

# What Survives, What Expires

| Idea from this lecture | On an HDD | On an SSD |
|---|---|---|
| Seek + rotation dominate | **the whole story** | gone entirely |
| Sequential ≫ random | 100–200× | ~2–10× — much weaker, but still real |
| Disk scheduling (SCAN, C-SCAN) | essential | **pointless** — Linux uses `none` for NVMe |
| Keep related data physically close | essential | no longer about seeks |
| Polling vs. interrupts | interrupts win | **reversed** — polling often wins |
| Caching in memory | essential | essential |
| Crash consistency | essential | essential |

- Mechanisms encode assumptions about hardware. Hardware changes. The assumptions expire.

# Lecture 1 Summary

### Going down: how a request travels

- `read(fd, ...)` → trap into the kernel
- **fd table** (in the PCB) → **open file table** (the offset) → the file's record
- File system → **generic block layer** *(device-neutral)*
- **Device driver** *(device-specific)*
- Status / command / data **registers**
- The device

### Coming back: who waits, and how

- **Polling** vs. **interrupts** — decided by device speed
- **PIO** vs. **DMA** — who moves the bytes
- **Coalescing** to survive high interrupt rates; **polling** to survive livelock
- Blocking I/O is just **Running → Blocked → Ready** from Topic 1

- And underneath it all: mechanical motion costs milliseconds; everything else is noise — until the mechanics go away
- **Next lecture:** we open the black box. What *is* that record the open file table points at?

# File System

- File system is responsible for organizing, storing, and managing files in a reliable and efficient manner on a storage device (disk)
- The most visible aspect of an OS
  - Files
  - Directories
  - Protections
  - Persistence
  - Transparent remote access

# File

- A logical, linear array of bytes stored on a storage device (disk)
- From programmer's view, a collection of data records
- From OS's standpoint, a bunch of blocks stored on disk

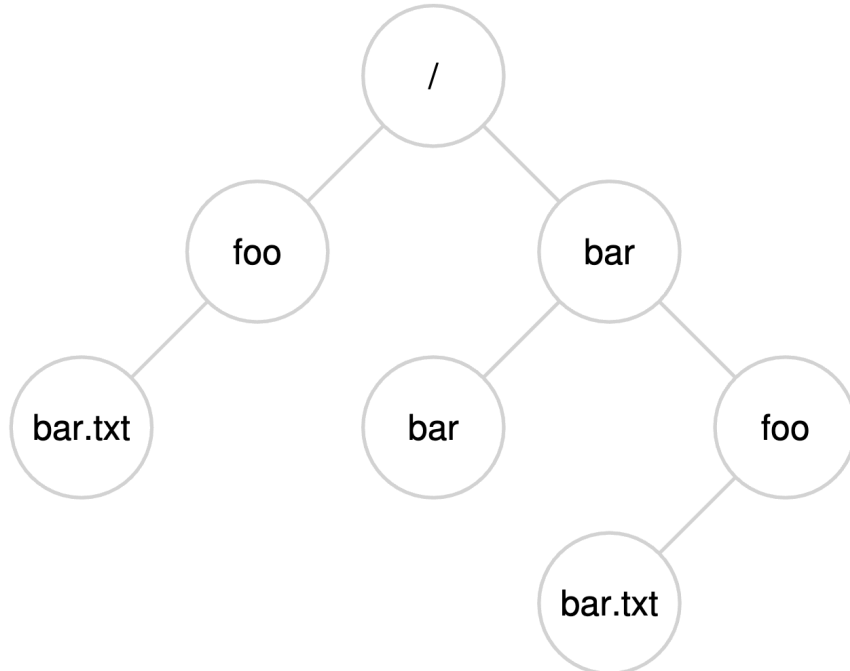
- File system performs the translation
  - Pack bytes into disk blocks on writing
  - Unpack them again on reading

# Why File?

- File system abstraction
  - Byte oriented
  - Named files
  - Users protected from each other
  - Robust to machine failures
- Physical reality (on disks)
  - Block oriented
  - Physical sector numbers
  - No protection among users
  - Data might be corrupted if machine crashes

# Directory

- A file system cataloging structure that contains references to other files and/or directories
  - A specially-formatted file
- Both and files and directories reside on disks
- A directory tree or directory hierarchy organizes all files and directories into a large tree, starting at the root



# What Makes File Systems Challenging?

- Files grow and shrink
  - Little a prior knowledge
  - 6-8 orders of magnitude in file sizes
- Overcome disk performance behavior
  - Highly nonuniform access patterns
    - Sequential: Information is processed in order, one piece after another
    - Random (direct): Access any block in the file directly
    - Content-based: Search for blocks with particular values
  - Desire for efficiency
- Cope with failures

# File System Components

- Naming and access: User gives file name to locate data
- Disk management: Arrange collection of disk blocks into files
- Security: Keep information secure
- Reliability: When system crashes, lose stuff in memory but want files to be durable

# Disk Layout of File System

- Disk is divided into blocks, e.g., 512 B and 4 kB
- Note that sectors are the smallest *physical* storage units on a disk (typically 512 bytes or 4KB), fixed by hardware
- Blocks (or clusters) are the smallest logical unit used by OS to manage data, composed of one or more contiguous sectors



# On-Disk File System Layout

- A file system divides the disk into fixed-size regions
  - **Superblock**: metadata about the whole file system
  - **Inode bitmap**: one bit per inode slot — 1 = allocated, 0 = free
  - **Data bitmap**: one bit per data block — 1 = allocated, 0 = free
  - **Inode table**: array of fixed-size inode structures
  - **Data region**: actual file and directory content

- Layout is fixed at format time → each region is at a known disk offset
  - O(1) to locate any inode or data block from its number

# Boot Block & Superblock

- Boot block: Contain info to boot OS
  - Bootstrap program (stored at fixed location)

- Superblock: Define a file system (size, layout, status)
  - Size of the file system
  - Free metadata (inode) count and pointers
  - Free data block count and pointers
  - Location of the metadata of the root directory

# The Superblock

- The **superblock** is the first block read when a file system is mounted
  - Contains: magic number (identifies FS type), total inode count, total data block count, start offset of inode table, FS-specific flags
- Magic number lets the OS verify it is reading the right kind of file system
- If the superblock is corrupted, the entire file system is unreadable
  - FFS (and ext2/3/4) replicate the superblock in each cylinder group for redundancy

# Inode (Index Node)

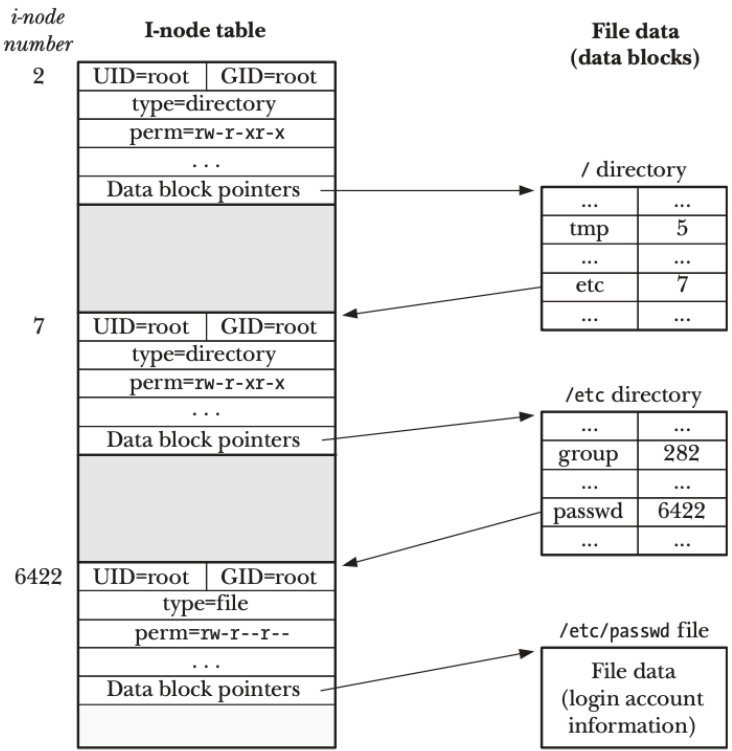
- Store metadata, which bookkeeps data that describes file or its info
  - Not actual content of the file

# What Is Inside an Inode?

- An **inode** is a fixed-size structure (typically 128 or 256 bytes) containing all file metadata except the name
  - **File type**: regular file, directory, symbolic link
  - **Size**: in bytes
  - **Link count**: number of directory entries pointing to this inode
  - **Permissions**: read/write/execute for owner, group, others
  - **Timestamps**: `atime` (last access), `mtime` (last data modification), `ctime` (last metadata change)
  - **Data block pointers**: direct, indirect, double-indirect, triple-indirect

- The file *name* is stored in a directory — not in the inode itself

# Inode Table & File Data Blocks




Source: https://www.lincolnbasnet.com.np/linux/Inode/

# Inode (Cont’d)

- Store metadata, which bookkeeps data that describes file or its info
  - Not actual content of the file

- Directory
  - Inode: Set flag (file type) indicating it is a directory
  - Data blocks: Contain mapping of <file/directory name, inode #>

# Inode Number: The File’s True Name

- Every file has a unique **inode number** (i-number) assigned by the OS
  - Like a PID, but for files — the low-level name the OS uses internally
  - The human-readable path is just a label pointing to this number
- Given an inode number, the OS computes the disk offset directly:
  - `disk offset = inode_table_start + (inode_number × inode_size)`
  - O(1) lookup — no searching required
- `stat` shows the inode number; `ls -i` prints it next to the file name

# Inode Number and Inode Lookup

- Every file has a unique **inode number** (i-number) — the low-level file name
  - The i-number is returned by `stat`, `ls -i`, and used internally by the kernel

- To find an inode on disk from its i-number:
  1. Multiply i-number by `sizeof(inode)` to get an offset within the inode table
  2. Add the inode table's start byte on disk
  3. Issue one disk read of `sizeof(inode)` bytes
- This is O(1): no searching, no linked-list traversal
  - Same idea as indexing into an array

# Hard Link VS. Soft Link

- Hard link points directly to the file's data (inode)
- Soft link (symbolic link) is a separate file that points to the name or path of the original file
- What is inode number of original/linked file?
- What will happen if original file is deleted?
- Why can hard link not link to directories?

# When Is a File Actually Deleted?

- `unlink(path)` removes **one directory entry** — it does not immediately delete data
- The inode tracks a **link count**: decremented each time `unlink()` is called

- File data is freed only when **both** conditions are true:
  1. Link count reaches **0** (no more directory entries point to the inode)
  2. No process has the file **open** (no open file descriptors remain)

- This is why the syscall is called `unlink` — it removes a link, not the file itself

# File System Interfaces

- `fd = open(<file name>, <mode>);`
  - Mode: O_CREAT, O_RDONLY, O_WRONLY, O_RDWR, etc.

- `read(fd, <user buffer>, size);`
- `write(fd, <user buffer>, size);`
- `close(fd);`
- `lseek(fd, <offset>, <whence>);`
  - Whence: SEEK_SET; SEEK_CUR; SEEK_END

# fsync() and Durability

- `write()` places data in the OS **buffer cache** — it may not reach disk immediately
  - The OS batches disk writes for performance
  - A crash between `write()` and the actual disk write **loses data**
- `fsync(fd)` forces all buffered data for `fd` to **persistent storage** before returning
  - Guarantees the data survives a power loss or crash
- `fsync()` is critical for any application that cannot afford to lose data
  - Databases, write-ahead logs, configuration updates

# rename(): Atomic File Update

- `rename(old_path, new_path)` renames or moves a file **atomically**
  - If `new_path` already exists, it is replaced in a single atomic step
  - No process sees an intermediate state where neither file exists

- Common pattern for safe file update:
  1. Write new content to a **temporary file** (e.g., `file.tmp`)
  2. Call `fsync()` to ensure data is durable
  3. Call `rename("file.tmp", "file")` to atomically swap
- Avoids the window where a crash could leave a partially-written file visible

# fd=open()

- File name lookup and authentication
  - Goal: Get inode # for the file
  - Input: Pathname (e.g., "/dir1/dir2/file1")
  - Algorithm
    - Fetch root inode
      - Start loading root directory data blocks into memory
      - Walk directory data until dir1 is found
      - Get inode # for dir1
    - Fetch dir1’s inode
      - Start loading dir1’s directory data blocks into memory
      - Walk directory data until dir2 is found
      - Get inode # for dir2

# fd=open() (Cont’d)

- File name lookup and authentication

- Create an entry in the open file table (if it is not in)
- Copy the file metadata to in-memory data structure (if it is not in)
- Create an entry in PCB
- Link up the data structures
- Return fd to user

# The I/O Cost of open()

- Opening `/foo/bar` requires **multiple disk reads** before any data is touched
  - Read root inode ( `/` ) → read root directory data block → find `foo`
  - Read `foo` 's inode → read `foo` 's directory data block → find `bar`
  - Read `bar` 's inode → open complete

- A **2-level path** requires **4+ disk I/Os** just to resolve the name
- Deeper paths cost even more: each additional directory level adds 2 I/Os
- This is why `open()` is expensive and why **caching** directory entries and inodes is critical

# Free Block Management: Bitmap

- Bitmap: 1 bit per block
- A bitmap for the inodes to correspond to allocated and unallocated inodes
- A bitmap for the blocks to correspond to allocated and unallocated blocks
- These bitmaps should take up at least one block each prior to the inode table.

# Disk Allocation Problem

- Definition: Allocate disk blocks when a file is created or grows, and free them when a file is removed or shrinks
- Two tasks
  - How to allocate blocks for a file?
  - How to design inode to keep track of blocks

- Challenge: File usage patterns
  - How do users access files?
  - How are files used?

- Cannot do anything about # of I/Os Focus on performance of I/Os

# Design Goals & Expectations

- Optimize I/O performance
  - What can be optimized for different access patterns?
- Minimize metadata size
  - Ideally fit in inode and track data blocks

# Why Multi-Level Indexing? Most Files Are Small

- File system design is driven by real-world workload measurements
  - **Median file size**: ~2 KB (half of all files fit in less than 2 KB)
  - **Average file size**: ~200 KB (pulled up by a few large files)
  - **Most bytes on disk**: stored in a small number of large files

- Key design insight: **optimize the common case**
  - Small files should be fast and cheap to access — no extra disk reads for indirection
  - Large files must still be supported, even if access is slightly slower

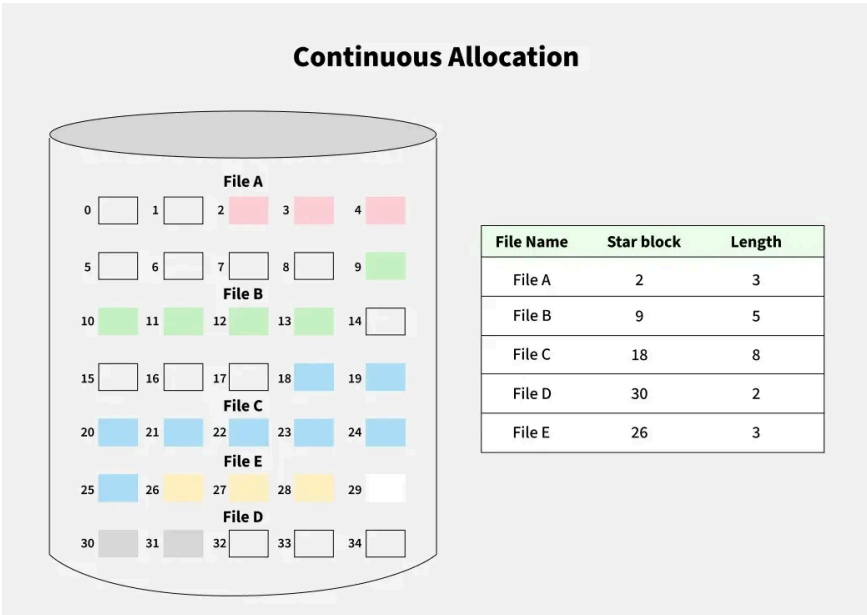
- Multi-level indexed structure reflects this:
  - 12 direct pointers cover up to 48 KB with zero indirection
  - Indirect and double-indirect pointers extend capacity for large files

# Disk Allocation Methods

- Contiguous
- Single-level indexed
- Linked
- Multi-level indexed (UNIX)

# Contiguous Disk Allocation

- Request in advance for the size of the file
- Search bitmap to locate a space
- File header stores
  - Address of starting block
  - Length of the allocated portion
- Pros
  - Fast sequential access
  - Easy random access
- Cons
  - External fragmentation
  - Hard to grow files

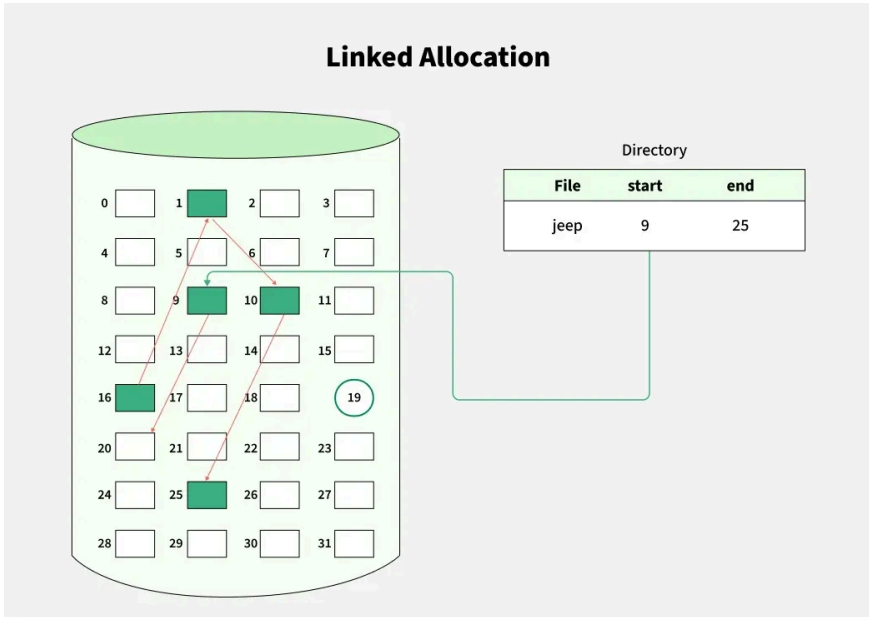



| File Name | Star block | Length |
|---|---|---|
| File A | 2 | 3 |
| File B | 9 | 5 |
| File C | 18 | 8 |
| File D | 30 | 2 |
| File E | 26 | 3 |

Adapted from https://www.geeksforgeeks.org/operating-systems/file-allocation-methods/

# Linked Files

- File header points to 1st block on disk
- Each block points to next
- Pros
  - Can grow files dynamically
  - No external fragmentation
  - Sequential access easy
- Cons
  - Random access is horrible
  - Unreliable: Losing a block means losing the rest




Adapted from https://www.geeksforgeeks.org/operating-systems/file-allocation-methods/

# Single-Level Indexed File

- A user declares max size
- A file header holds an array of pointers to point to disk blocks
- Pros
  - Can grow up to a limit
  - Random access is fast
  - No external fragmentation

- Cons
  - Clumsy to grow beyond limit
  - Up-front declaration
  - File header cannot fit in inode
    - Large inode would be wasteful

Adapted from https://www.geeksforgeeks.org/operating-systems/file-allocation-methods/

# Multiple-Level Indexed File

- 13/15 pointers in a header
  - 10/12 direct pointers
  - 1 1-level indirect
  - 1 2-level indirect
  - 1 3-level indirect

- Pros/Cons
  - In favor of small files
  - Can grow
  - Random access has up to 4 seeks

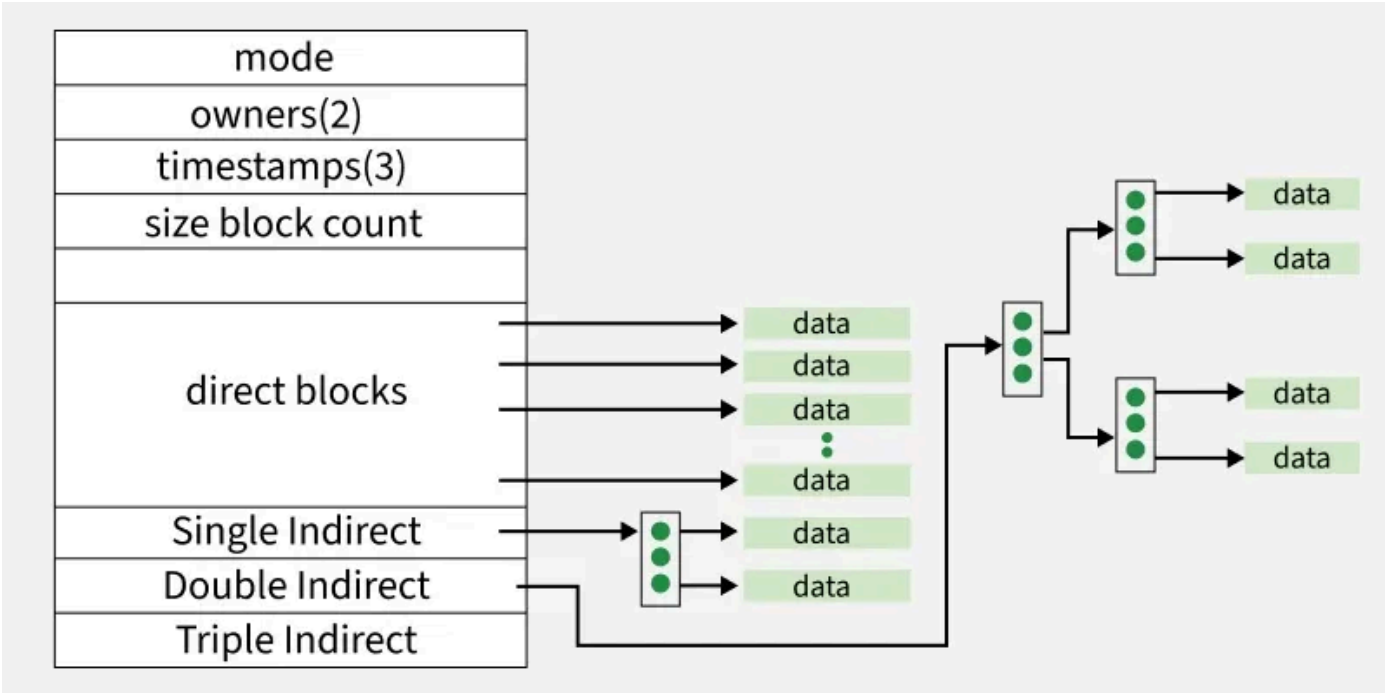
Adapted from https://www.geeksforgeeks.org/operating-systems/file-allocation-methods/

# Multiple-Level Indexed File: A Worked Example

- What is maximum file size (measured in number of blocks) on this Unix system?

- 12 direct pointers
- 1 1-level indirect
- 1 2-level indirect
- Disk addresses are 32 bits
- Block size is 4 kB

# Multiple-Level Indexed File: Access Cost



# Multi-Level Index: Capacity Numbers

- Assume **4 KB blocks** and **4-byte pointers** (1024 pointers per indirect block)
- Each pointer level reaches exponentially more data:

| Pointer type | Blocks reachable | Max file size added |
|---|---|---|
| 12 direct | 12 | 48 KB |
| 1 indirect | 1,024 | +4 MB |
| 1 double-indirect | $1{,}024^2$ | +4 GB |
| 1 triple-indirect | $1{,}024^3$ | +4 TB |

- Design is **biased toward small files**: no indirection needed for files ≤ 48 KB
  - Most files are small (median ~2 KB in practice)

# Extents: An Alternative Pointer Structure

- An **extent** stores a contiguous run of blocks as `(start_block, length)`
  - More compact than per-block pointers when a file is laid out contiguously
  - One extent replaces hundreds of individual block pointers
- Inode holds a small array of extents instead of an array of block numbers
  - If the file fits in a few extents, no indirect blocks are needed at all
- Used by modern file systems: **ext4**, **XFS**, **Btrfs**
- Trade-off: works well for sequentially laid-out files; degrades to many extents for fragmented files

# Directories: Files Containing Name-to-Inode Mappings

- A **directory** is stored on disk as a regular file — it has its own inode
  - Its data blocks contain a list of **directory entries**

- Each directory entry maps a human-readable name to an inode number
  - Structure: `{inode_number, record_length, name_length, name[]}`
  - Record length allows skipping over deleted entries without compacting

- Special entries: `.` (this directory) and `..` (parent directory)
- Path traversal requires reading directory data blocks at each level
  - `/foo/bar` → read `/` directory data → find `foo` inode → read `foo` directory data → find `bar` inode → read `bar` inode → read data blocks

# How to Optimize File Name Lookup?

- Key idea: Caching (due to lots of common lookups)
  - Keep cache of name lookups
  - Keep cache of recently-used inodes, dir blocks
  - Reserve some kernel memory for this

- Principle of locality
  - Locality of reference in file accesses
  - Keep a number of disk blocks in "much faster" memory
    - When accessing disk, check cache first!

# Read Operations w/ Caching

- On a hit, copy from file buffer cache to a user buffer
- On a miss
  - Replace if necessary
  - Read a file block into file buffer cache

# Write Operations w/ Caching

- On a hit, write to file buffer cache

- On a miss?
  - Allocate a new cache block, load the old block from disk (if needed), then write
  - Mark the cache block as **dirty** — it must be flushed to disk eventually

- **Write-back caching**: dirty blocks are written to disk asynchronously
  - Better performance (batches multiple writes into one disk operation)
  - Risk: data in the cache can be **lost on a crash** before flush

# File Persistence under Caching

- Problem: Fast cache memory is volatile, but users expect disk files to be persistent
  - In the event of a system crash, dirty blocks in file buffer cache are lost!

- Example 1: Create a new file "/dir/a"
  - Allocate inode (from free inode list) for the file
  - Update parent directory content, i.e., add <"a", inode #> to "dir"

- Example 2: Append a block to a file
  - Allocate a new data block from free block list
  - Update inode content
  - Write to the new data block

# Why Writes Are Dangerous: Multiple Structure Updates

- A single `write()` that appends a new data block touches **three** on-disk structures:
  1. **Data block** — the new content
  2. **Inode** — updated file size, new block pointer, new modification time
  3. **Data bitmap** — mark the new block as allocated
- If the system crashes **between** any two of these writes, the disk is in an **inconsistent state**
  - e.g., bitmap says block is allocated, but inode has no pointer to it → space leaked
  - e.g., inode points to block, but bitmap says it is free → double allocation possible
- This is the **crash consistency problem** — solved by journaling

# Solution 1: Write-Through Cache

- Modifications are written to disk immediately
- No performance advantage for disk writes

# Solution 2: Limit Potential Data Loss

- Write-through caching for metadata (inodes, directories, free block list)
- Write back (deferred write)
  - Dirty data blocks after no more than 30 seconds
  - All dirty blocks during file close

# Other Performance Optimizations

- Read-ahead (e.g., Linux): For sequential access, read the requested block and the following N blocks together
- Write-behind: Start disk write, but do not make applications wait until the disk operation completes
- Allow overlap of a process's computation with its own disk I/O

# The Old UNIX File System: A Performance Disaster

- The original UNIX file system used only ~**2% of available disk bandwidth**
- Root cause: data placed with no regard to disk geometry
  - Inode table at one end of disk, data blocks at the other → every file access = long seek
  - As files are created and deleted, the **free list** becomes fragmented
  - Sequential file bytes end up in scattered non-adjacent blocks → many short seeks
- Problem compounds over time
  - A freshly formatted disk works reasonably well
  - After months of use, files fragment across the disk → performance degrades
- **Key insight**: disk seeks are expensive; placing related data physically close eliminates them

# Fast File System (FFS): Cylinder Groups

- FFS (Berkeley, 1984) divided the disk into **cylinder groups** (also called block groups)
  - Each group is a consecutive range of disk blocks managed as a self-contained unit

- Each cylinder group contains its own copies of:
  - **Superblock copy** — redundancy, can mount from any group if primary is corrupted
  - **Group descriptor** — tracks free inode/block counts for this group
  - **Inode bitmap** — which inodes in this group are free/allocated
  - **Data bitmap** — which data blocks in this group are free/allocated
  - **Inode table** — inodes for files in this group
  - **Data blocks** — actual file and directory data

- Keeping inodes and data blocks in the same group eliminates the long seek between them
  - ext2, ext3, ext4 all use FFS-style block groups

# FFS: Placement Policy — Keeping Related Data Together

- Core principle: **keep related data in the same cylinder group**
- Placement rules:
  - **New directory**: place in the group with the fewest existing directories and most free inodes (load balance across groups)
  - **File in a directory**: place in the same group as the directory's inode
  - **File's data blocks**: place in the same group as the file's inode
  - **Large file exception**: after allocating ~1 MB in one group, switch to a different group to avoid filling one group entirely
- Effect: files in the same directory are physically close → directory listing reads are fast
- Spreading large files prevents any single group from becoming a bottleneck

# FFS: Large Block Size and Sub-Blocks

- FFS increased the default block size from **512 bytes to 4 KB (or 8 KB)**
  - Larger blocks mean more bytes transferred per disk rotation → higher throughput
  - Fewer pointer entries needed in inodes and indirect blocks

- Problem: **internal fragmentation** for small files
  - A 1-byte file wastes 4 KB − 1 byte = 4095 bytes of disk space
  - With median file size ~2 KB, internal fragmentation is significant

- FFS solution: **sub-blocks**
  - Small files start in 512-byte sub-blocks within a larger block
  - When a file grows to fill a complete 4 KB block, sub-block contents are copied into the full block
  - Sub-blocks are reclaimed and reused

- This trades some copy overhead for much better space utilization for small files

# What Else FFS Introduced

- FFS was not just a performance improvement — it also added features that remain standard today
  - **Long file names**: up to 255 characters (original UNIX limited to 14)
  - **Symbolic links**: a new file type that stores a path string instead of data
  - **Atomic rename**: `rename()` system call replaces destination atomically
  - **Advisory locks**: `flock()` for cooperative file locking between processes
- These features defined the POSIX file system interface still used today

# File System Reliability

- Three threats
  - Accidental/malicious deletion of data
  - Disk failure
  - System crash during file system modifications
- Solutions
  - Backup
  - Disk mirroring
  - Consistency

# Backup

- Copy entire file system onto low-cost media at regular intervals (e.g., once a day)
  - Implementation: Do we need to copy the whole FS?

- In the event of a disk failure, replace disk and restore from backup media
- Amount of loss is limited to modifications occurred since last backup

# Mirrored Disks

- Multiple copies of file system are maintained on independent disks
- Disk writes update all redundant disks in parallel
- RAID (Redundant Array of Independent Disks) Use multiple parallel disk drives for higher throughput and increased reliability
- Can recover data if 1 disk fails

# Redundant Array of Independent Disks

- RAID (Redundant Array of Inexpensive/Independent Disks): Transform a number of independent disks into a large, more capacious, and more reliable single entity
  - Transparency: Hardware and software above RAID is relatively oblivious to it

- Motivations
  - Operating in parallel can increase disk throughput
  - Redundancy can increase reliability

# Idea 1: Bit/Block-Level Stripping

- Split bits of a byte across multiple disks (e.g., 8)
  - Logically act like single disk

- Each 8-bit byte would be read in parallel by 8 heads on separate disks
- Alternatively, block-level stripping
  - Block N is located on disk 0, then block N + 1 will be on disk 1, and so on

# Idea 2: Mirroring

- Local disk consists of 2 physical disks in parallel
- Every write is performed on both disks
- Can read from either disk

# Synopsis of RAID Levels

- RAID Level 0: Stripping only
- RAID Level 1: Mirroring only
- RAID Level 2: Bit-interleaved + Error Correction Code (ECC)
  - 3 additional disks to protect 4 disks
    - # of disks required is a function of the error-correcting algorithms
- RAID Level 3: Byte-interleaved + Parity
- RAID Level 4: Block-interleaved + Parity
- RAID 5: Block-interleaved + Distributed parity

# Crash Recovery: Journaling (Write-Ahead Logging)

- Goal: Make sure on-disk data is always in a consistent state
  - if a crash occurs, you may lose some work, but the file system structures on disk will be in a consistent state

- Key idea: Before updating the disk, first write down a little note (in the journal on disk) describing what you are about to do
  - Update metadata and, optionally, file data transactionally
    - "all or nothing", i.e., atomically

  - Write a "journal" of updates you intend to make before attempting the updates themselves
    - After a crash, quickly get it to a consistent state by using the transaction log/journal

- Transaction: A set of changes for performing a specific task

# Pre-Journaling Solution: fsck

- `fsck` (File System Check): scan the entire file system on boot to detect and repair inconsistencies
  - Checks that every inode's block pointers match the allocation bitmap
  - Verifies directory structure, link counts, and free lists

- Problem: **very slow** — must read every inode, block, and directory on the disk
  - Minutes to hours on large disks; unacceptable for modern systems

- Also **not complete**: if only the data block was written (no metadata), `fsck` sees a consistent state — but data may be wrong
- Journaling replaces `fsck` as the primary crash recovery mechanism
  - `fsck` still runs, but only as a last resort; journaling limits recovery to replaying the journal

# Journal Structure: TxB, Content, TxE

- A journal entry contains three parts written in order:
  1. **TxB (Transaction Begin)**: marks the start; contains the transaction ID and which blocks will be updated
  2. **Updated blocks**: copies of the inode, bitmap, and (optionally) data that are about to change
  3. **TxE (Transaction End)**: written last; its presence on disk is the **commit point**
- Writing TxE is the point of no return
  - Before TxE reaches disk → transaction is incomplete; safe to discard on recovery
  - After TxE reaches disk → system guarantees the updates will eventually be applied

- After commit, the actual disk locations are updated (**checkpointing**)
- After checkpointing, the journal entry is freed (journal is a **circular buffer**)

# Crash Recovery: Replaying the Journal

- On boot after a crash, the OS scans the journal for committed but not-yet-checkpointed entries
  - **Committed** = TxB and TxE are both present in the journal
  - **Incomplete** = TxB present but no matching TxE → discard (crash happened before commit)
- For each committed entry, **replay** (redo) the updates to their real disk locations
  - This is safe to do multiple times — replaying a committed entry that was already checkpointed is harmless
- Recovery is fast: only the journal is scanned, not the entire disk
  - Contrast with `fsck`: journaling limits recovery time to seconds rather than hours

# Journaling Modes: Data, Ordered, & Writeback

- **Data journaling** (safest)
  - Journal: inode + bitmap + data blocks
  - Every write goes to disk twice (journal then real location)
  - Best protection; slowest performance

- **Ordered journaling** (ext3/ext4 default)
  - Journal: inode + bitmap only (metadata)
  - Data blocks written to final location **before** committing journal entry
  - If crash before data write → old data survives; metadata not committed → no inconsistency

- **Writeback mode** (fastest, weakest)
  - Journal: metadata only, data written in any order
  - Crash may expose stale or garbage data in a new file
  - Metadata consistency guaranteed; data content is not

# Journaling Modes: Comparison

| Mode | Journaled | Performance | Safety |
|---|---|---|---|
| Data | All | Slowest | Best |
| Ordered | Metadata | Medium | Good |
| Writeback | Metadata | Fastest | Weakest |

# Disk Failure Modes: Beyond Fail-Stop

- **Fail-stop model** (traditional assumption): the whole disk fails completely and detectably
  - RAID handles this: rebuild from parity or mirror

- Modern disks exhibit **fail-partial** behavior — not just total failure
  - **Latent Sector Error (LSE)**: one sector becomes unreadable; disk returns an explicit error
    - Detectable; fixed by RAID redundancy (use another copy)
  - **Silent corruption (block corruption)**: disk returns wrong data with no error signal
    - Disk's internal ECC corrected the data — to a wrong value
    - Undetectable without a checksum

- A study of 1.5 million drives over 3 years found:
  - ~9% of cheap (SATA) drives had at least one LSE; ~0.5% had silent corruption

# Latent Sector Errors vs. Silent Corruption

- **Latent Sector Error (LSE)**
  - Disk sector physically damaged or ECC-unrecoverable
  - Disk returns an **explicit error code** on read
  - Handled by RAID: reconstruct the block from parity or mirror

- **Silent corruption**
  - Disk stores wrong bits but returns them **without any error signal**
  - Cannot be detected by RAID alone — RAID will happily serve corrupted data
  - Detected only by comparing a **stored checksum** against a recomputed one

- Key distinction: LSE = "disk tells you"; corruption = "disk lies to you"
  - Different failure modes require different solutions

# Checksum-Based Data Integrity

- **Checksum**: a compact digest of a data block stored alongside the block
  - Common algorithm: **CRC-32** (cyclic redundancy check) — fast, detects most bit-flip patterns
  - Others: Fletcher checksum, Adler-32, MD5/SHA-1 (slower, used for security)

- On write: compute checksum of data; store it in a separate metadata area
- On read: recompute checksum of returned data; compare to stored checksum
  - Match → data is intact; Mismatch → **corruption detected** → use RAID copy or replica

- **Misdirected write detection**: include the block's **physical block address (PBA)** in the checksum
  - If a block is written to the wrong disk location, its checksum will not match when read back from the correct location

- Placement: checksum stored in a separate per-block metadata area (ext4) or packed into the block (ZFS)

# Lost Writes and Disk Scrubbing

- **Lost write**: disk acknowledges a write (`write()` returns success), but data never persisted
  - Caused by write-back caching bugs or controller failures
  - Checksum alone does not detect this: the old (stale) data has a valid old checksum
  - Detection: store a **write timestamp** or version number in the checksum metadata; stale data will have an outdated timestamp

- **Disk scrubbing**: periodically read every block and verify its checksum
  - Proactively finds latent sector errors and silent corruption before they compound
  - Critical for RAID: an undetected LSE on one disk + a second disk failure during reconstruction = unrecoverable data loss
  - Trade-off: scrub too rarely → errors accumulate undetected; scrub too often → wears SSDs, consumes bandwidth
  - Typical practice: scrub every few weeks to months in production systems